

Introdução à Computação

árvores binárias de busca

Alexandre Rademaker

árvores I

Uma árvore é um tipo de dados que representa um conjunto de nós conectados em uma estrutura hierárquica.

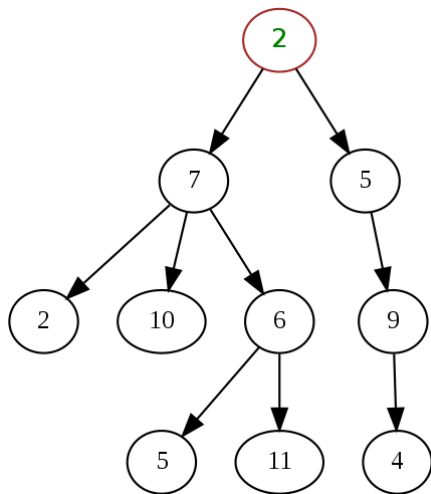
Cada nó de uma árvore pode ser conectado a um número qualquer de **filhos** - dependendo do tipo de árvore - mas apenas a um **pai**. Um único nó, chamado **raiz**, não tem pai.

Isto significa que árvores não podem conter **ciclos** e que cada nó pode ser considerado raiz de uma sub-árvore.

Com isso, **recursão** é uma técnica útil para travessia da árvore.

árvores II

- ▶ raiz
- ▶ folhas
- ▶ sub-árvores
- ▶ grau
- ▶ nível
- ▶ altura



árvores III

Árvores binárias são um tipo de árvore onde cada nó pode conter até dois filhos.

Quando a ordem dos filhos é relevante, chamamos de **árvore ordenada**.

Valores podem ser associados a cada nó na árvore ou, as vezes, apenas aos nós **folhas** - nós que não tem filhos.

árvores IV

aplicações:

- ▶ sistema de arquivos dos sistemas operacionais
- ▶ DOM de páginas HTML e arquivos XML
- ▶ árvores de busca para armazenamento de dados de forma a permitir buscas eficientes.
- ▶ árvores sintáticas no processamento de linguagem natural.

árvores V

Um **tipo abstrato de dados** pode ser representado de diferentes formas.

Na memória, tipicamente nós são alocados dinamicamente com ponteiros para os filhos, pais, ou ambos, além dos dados associados ao nó.

Se o número de filhos é variado, a cada nó podemos usar um array ou lista encadeada para armazenar ponteiros para os filhos.

Também podemos representar a árvore com duas listas, uma para os nós e outra para as relações pai-filho (lista de adjacências), ou um array onde os relacionamentos (pai-filho) são determinados pelas posições no array.

árvores VI

```
1 '(2 (7 (2) (10) (6 (5) (11))))  
2   (5 (9 (4))))
```

Se nossa implementação de lista encadeada permitisse que elementos da lista fossem de qualquer tipo, incluindo listas...

também poderíamos implementar uma árvore como uma lista de listas, onde o primeiro elemento de cada sublista é a raiz da sub-árvore.

Para saber [mais!](#)

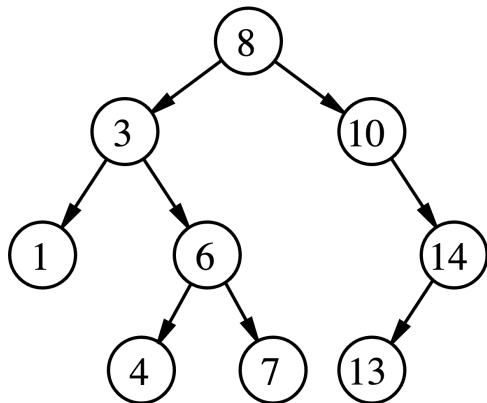
árvores binárias de busca I

Uma árvore binária de busca, também chamada de árvore binária ordenada, é uma árvore binária onde o valor de cada nó interno é maior que todos os valores na subárvore esquerda do respectivo nó e menor que as de sua subárvore direita.

A complexidade de tempo das operações na árvore de busca binária é diretamente proporcional à altura da árvore.

Buscas binárias podem ser facilmente implementadas em árvores binárias de busca.

árvores binárias de busca II



árvores binárias de busca III

Em C, os nós precisam ser agora declarados com

tree.c

```
1 typedef struct node {  
2     int key;  
3     struct node *left, *right;  
4 } t_node;
```

árvores binárias de busca IV

Para inserir um novo elemento, uma função auxiliar que cria um novo nó:

tree.c

```
1 t_node* new_node(int item) {  
2     t_node* t = malloc(sizeof(struct node));  
3     assert(t != NULL);  
4     t->key = item;  
5     t->left = t->right = NULL;  
6     return t;  
7 }
```

Notação $a = b = \text{expressão!}$

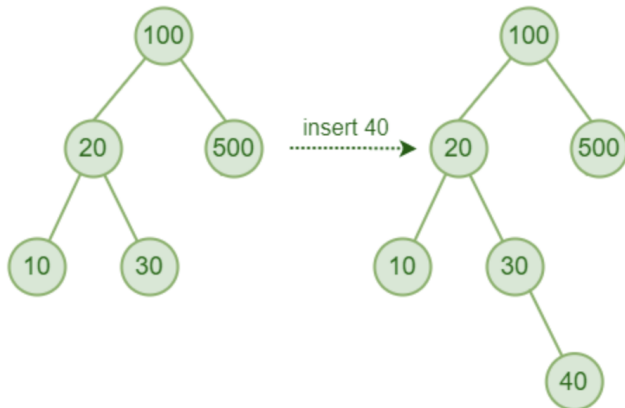
árvores binárias de busca V

Agora a inserção, seja o nó corrente com valor V, devemos comparar com o valor X sendo inserido. Se X menor, movemos para a árvore à esquerda. Se X maior, para árvore da direita.

tree.c

```
1 t_node* insert(t_node* node, int key) {
2     if (node == NULL)
3         return new_node(key);
4
5     if (key < node->key)
6         node->left = insert(node->left, key);
7     else if (key > node->key)
8         node->right = insert(node->right, key);
9
10    return node;
11 }
```

árvores binárias de busca VI



árvores binárias de busca VII

Nós criados de forma dinâmica precisam ser liberados.

tree.c

```
1 void free_tree(node *root) {  
2     if (root == NULL)  
3         return;  
4  
5     free_tree(root->left);  
6     free_tree(root->right);  
7     free(root);  
8 }
```

busca binária I

Usando esta estrutura, podemos então realizar buscas

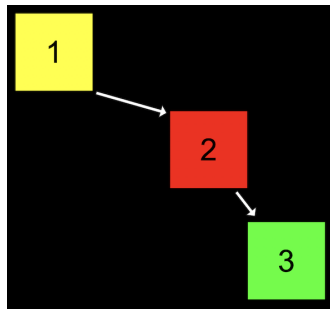
tree.c

```
1 bool binary_search(t_node *root, int n) {  
2     if (root == NULL)  
3         return false;  
4     else if (n < root->key)  
5         return binary_search(root->left, n);  
6     else if (n > root->key)  
7         return binary_search(root->right, n);  
8     return (n == root->key);  
9 }
```

busca binária II

Mas temos que 'balancear' a árvore, para não terminar com uma estrutura ineficiente.

Em uma árvore balanceada, busca e inserção são $O(\log n)$, mas em uma árvore não balanceada, pode ser $O(n)$.



rebalanceamento “em batch” I

Uma primeira abordagem: depois de carregar todos os elementos, achatamos a árvore em um array ordenado (basta um **inorder**) e reconstruímos tomando o elemento do meio como raiz, recursivamente.

Custo $O(n)$, mas é uma operação ocasional. Para `sorted50000.txt` a altura cai de 50000 para $\lceil \log_2 50000 \rceil \approx 16$.

rebalanceamento “em batch” II

tree.c

```
1 int size(t_node* root) {  
2     if (root == NULL) return 0;  
3     return 1 + size(root->left) + size(root->right);  
4 }  
5  
6 void to_array(t_node* root, t_node** arr, int* i) {  
7     if (root == NULL) return;  
8     to_array(root->left, arr, i);  
9     arr[(*i)++] = root;  
10    to_array(root->right, arr, i);  
11 }
```

rebalanceamento “em batch” III

tree.c

```
1 t_node* from_sorted(t_node** arr, int lo, int hi) {
2     if (lo > hi) return NULL;
3     int mid = (lo + hi) / 2;
4     t_node* root = arr[mid];
5     root->left  = from_sorted(arr, lo, mid - 1);
6     root->right = from_sorted(arr, mid + 1, hi);
7     return root;
8 }
```

rebalanceamento “em batch” IV

tree.c

```
1 t_node* balance(t_node* root) {  
2     int n = size(root);  
3     t_node** arr = malloc(n * sizeof(t_node*));  
4     int i = 0;  
5     to_array(root, arr, &i);  
6     t_node* novo = from_sorted(arr, 0, n - 1);  
7     free(arr);  
8     return novo;  
9 }
```

Observe que `to_array` e `from_sorted` **reaproveitam** os mesmos nós; não há alocação de novos nós, só reorganização dos ponteiros.

AVL — balanceamento a cada inserção I

Outra abordagem: manter a árvore balanceada **durante** as inserções. Cada **insert** passa a custar $O(\log n)$ no pior caso, inclusive para entradas ordenadas.

Cada nó passa a armazenar a sua altura. Definimos o **fator de balanceamento** de um nó como a diferença entre as alturas das subárvores esquerda e direita. A árvore é dita AVL quando esse fator está em $\{-1, 0, +1\}$ para todo nó.

AVL — balanceamento a cada inserção II

tree.c

```
1 typedef struct node {  
2     int key;  
3     int height;  
4     struct node *left, *right;  
5 } t_node;
```

O campo `height` é **ignorado** pelas funções já vistas (`insert`, `height`, `binary_search`). Ele passa a ser usado apenas pelas funções a seguir.

AVL — balanceamento a cada inserção III

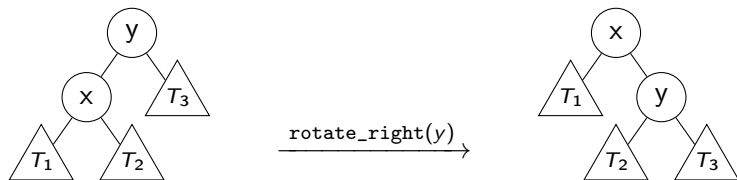
tree.c

```
1 int node_height(t_node* n) {
2     return n == NULL ? 0 : n->height;
3 }
4
5 int balance_factor(t_node* n) {
6     return node_height(n->left) - node_height(n->right);
7 }
8
9 void update_height(t_node* n) {
10     int l = node_height(n->left);
11     int r = node_height(n->right);
12     n->height = 1 + (l > r ? l : r);
13 }
```

AVL — rotações I

Uma rotação reorganiza localmente três nós, preservando a propriedade de árvore binária de busca — a sequência do **inorder** não muda — e podendo reduzir a altura local.

AVL — rotações II



Inorder antes e depois: T_1, x, T_2, y, T_3 — a propriedade de BST se preserva. rotate_left é simétrica.

AVL — rotações III

tree.c

```
1 t_node* rotate_right(t_node* y) {  
2     t_node* x = y->left;  
3     t_node* t = x->right;  
4     x->right = y;  
5     y->left = t;  
6     update_height(y);  
7     update_height(x);  
8     return x;  
9 }
```

AVL — rotações IV

tree.c

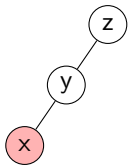
```
1 t_node* rotate_left(t_node* x) {
2     t_node* y = x->right;
3     t_node* t = y->left;
4     y->left  = x;
5     x->right = t;
6     update_height(x);
7     update_height(y);
8     return y;
9 }
```

AVL — inserção I

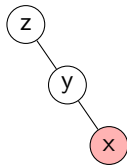
Depois da inserção recursiva, atualizamos a altura do nó e verificamos o fator de balanceamento. Se ele sair do intervalo $[-1, +1]$, aplicamos uma ou duas rotações — são quatro casos: LL, RR, LR, RL.

Seja z o ancestral mais próximo do nó recém-inserido (em vermelho) cujo fator de balanceamento ficou fora de $[-1, +1]$, e y o filho de z no lado mais alto:

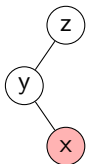
AVL — inserção II



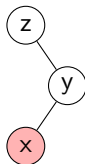
LL: `rotate_right(z)`



RR: `rotate_left(z)`



LR:
`rotate_left(y); rotate_right(z)`



RL:
`rotate_right(y); rotate_left(z)`

AVL — inserção III

tree.c

```
1 t_node* insert_avl(t_node* node, int key) {
2     if (node == NULL) return new_node(key);
3
4     if (key < node->key)
5         node->left = insert_avl(node->left, key);
6     else if (key > node->key)
7         node->right = insert_avl(node->right, key);
8     else return node;
9
10    update_height(node);
11    int bf = balance_factor(node);
```

AVL — inserção IV

tree.c

```
1 // LL
2 if (bf > 1 && key < node->left->key)
3     return rotate_right(node);
4 // RR
5 if (bf < -1 && key > node->right->key)
6     return rotate_left(node);
7 // LR
8 if (bf > 1 && key > node->left->key) {
9     node->left = rotate_left(node->left);
10    return rotate_right(node);
11 }
12 // RL
13 if (bf < -1 && key < node->right->key) {
14     node->right = rotate_right(node->right);
15     return rotate_left(node);
16 }
17
18 return node;
```

AVL — inserção V

Em main, basta trocar `insert` por `insert_avl` para que `sorted50000.txt` caia da altura 50000 para ≈ 16 .

AVL — exemplo I

Inserindo a sequência 1, 2, 3, 4, 5 em uma AVL inicialmente vazia.
Em **vermelho**, o nó recém-inserido; em **azul**, o ancestral onde $bf \notin [-1, +1]$ — ponto de rotação.

AVL — exemplo II

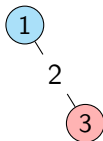
Inserções 1, 2 e 3:



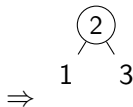
após ins. 1



após ins. 2



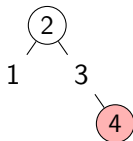
após ins. 3
 $bf(1) = -2$



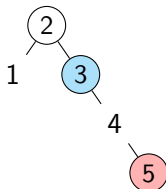
rebalanceada

AVL — exemplo III

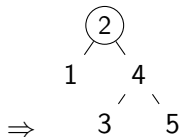
Continuando com 4 e 5:



após ins. 4



após ins. 5
 $bf(3) = -2$



rebalanceada

AVL — exemplo IV

Para a sequência ordenada $1, 2, \dots, n$, a árvore AVL termina com altura $\lceil \log_2 n \rceil + 1$ em vez de n .

Animações interativas e detalhes adicionais:

en.wikipedia.org/wiki/AVL_tree (Wikipedia traz figuras CC-BY-SA das rotações e dos quatro casos, além de provas de complexidade).